

Day 2 · 8/5

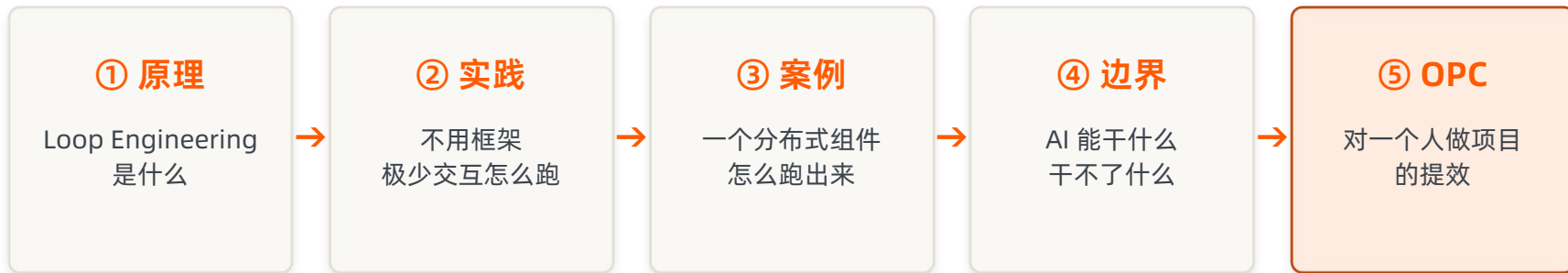
Loop Engineering 的实践， 与它给 OPC 带来的提效

不用框架，只用 SDD 与 Harness 思想，让 AI 自己把一个组件跑出来

以一个分布式组件为例 · 讲清怎么做、边界在哪、对一个人做项目意味着什么

企业级 AI 编程实战营 · 系列直播（二）

五段，从方法讲到自己的项目



今晚的核心结论：Loop Engineering 能把“执行”外包出去，极大提升一个人的产出；但判断力、思考和产品力，只能是你自己的。

提示：分布式组件只是例子，重点是这套方法怎么用在你自己的项目上。

一个人做项目，卡在哪

- 不是打字速度——有了 AI，写代码这一段确实快了很多。
- 真正卡人的是确定性：什么时候能交付、交付出来的东西对不对、要返工几次。
- 一个人没有 QA、没有 code review、没人帮你兜底，所以“怎么知道它是对的”成了最大成本。

Loop Engineering 解决的正是这件事：不是让 AI 写得更快，是让它在一个能自己判断对错的循环里干活，把确定性做出来。

Loop Engineering 是什么

一句话：设计一个让 AI 自主循环工作的环境和规则，让一件事在循环里做完 —— 而不是你坐着一条条下指令。

过去：你是驱动者

- 一条条下指令，来回拉扯
- 每步都要你看结果、给反馈
- 你在场，进度才动

现在：你是设计者

- 定义目标、判官、边界
- 循环自己做·测·改
- 你离开，它继续转

关键转变：精力从“写提示词”转到“设计环境和规则”。

一个循环由五个要素构成

① 可判定目标

干成什么算成功

能转

② 判官（验收）

测试/编译/CI 能判对错

能转

③ 隔离工作区

独立分支，失败不污染

安全

④ 边界与停止

最多几轮、何时交人

安全

⑤ 人的检查点

完成或卡住时给人看

可信

记住这个关系：前两个决定“能不能转”，中间两个决定“安不安全”，最后一个决定“可不可信”。缺一个，循环就跑不住。

最容易被跳过的是②：没有判官的循环，只是在高速地放大错误。

原理

一轮循环：做·测·改，直到全绿



修完回到“跑验收”，直到全绿或撞上停止条件

四条停止条件（撞任一条就停）：

验收全绿 → 交接等 review

超过轮数上限 → 停下报告卡在哪

输入文档有歧义 → 停下来问，不猜

要动边界外的代码 → 停下等确认

注意：“停下来问”比“猜着做”省得多——这条规则要写死。

我不用框架，只用两个思想

SDD 思想

先把问题定义清楚：做什么、边界在哪、什么算做完。
规格先行，不急着写代码。

Harness 思想

给 AI 搭一个能自己判断对错的工作台：代码规范、
安全红线、验收判官。

- 不装框架、不背命令、不搞复杂流程——这两个思想就够启动一个循环。
- 流程越轻，越容易真的用起来；工具越重，越容易半途放弃。

目标：用极少的交互，把一件事跑完。

极少交互：三步，中间不插手

① 定义清楚问题

这块要做什么、不做什么、什么算完



② 写清输入/输出文档

功能、接口、参数、验收、边界，一次写透



③ 交出去，让它跑

循环自转，直到全绿或停下问你

- 为什么要“极少交互”：每次插嘴都会打断它的上下文，也把你自己拖回执行层。
- 把话说在前面，比在过程中反复纠正省力得多——这是这套做法最反直觉的地方。

经验：文档写不清楚，说明问题还没想清楚，这时候开工必返工。

输入文档写什么：六块，20 分钟能写完

做什么（功能）

和谁交互（时序）

接口与参数

验收标准

必须有的测试

边界与禁止

两块橙色的最关键：验收和测试必须“可判定”——能自动判对错。写不出可判定的验收，说明这块还没想清楚，先别开工。

反面例子：“性能要好”“体验要顺”——这种验收循环没法用，等于没写。

Harness 搭什么：三面

代码

结构、命名、分层、lint -- 产出长成
一个样子

安全

不硬编码密钥、环境变量注入、不越
权

验收

单测 / 集成 / CI -- 循环唯一的判官

一句话： Harness 决定你返工几次。台子搭好，它自己能干很久；搭不好，你得一直盯着。

最省力的做法： 公共部分（安全红线、规范）写一次全局生效；每块只带自己的验收。

最要防的一件事：判官被收买

AI 会这样“骗过”验收

- 为了让测试变绿，改测试、删测试
- 把断言放松成“总能过”
- 悄悄跳过难场景（并发、边界）

护栏怎么立

- 写进规则：禁止改动测试与验收
- 实现与验证分开，互相制约
- 重的验证交 CI —— 它够不着

分层判官：快测试放循环里让它自我修正；慢的、破坏性的（杀节点、造故障）交 CI 和人跑。

提醒：不追覆盖率数字，追“能不能抓住关键失败模式”。

拿一个不好糊弄的东西试：分布式组件

- 为什么不用轻量例子：做个网站、做个小工具，证明不了什么。
- 选一个延时消息投递组件（When）：到点要准、不能丢消息、节点挂了要自动接管。
- 它同时有两类活——算法与接口（AI 很擅长），和分布式正确性（AI 最容易翻车）。

要说明的是：组件只是载体。你换成一个 SaaS、一个平台、一个工具，方法完全一样——只是判官换成别的形式。

案例

这个组件要做到什么

精度

到点触发要准，量大时更难

不丢

已受理的消息，机器挂了也必须投出去

高可用

Master 故障 10 秒内自动接管

可扩展

下游多种类型、量增长要能横向扩

这些就是判官的来源：每一条都能写成可自动判定的断言——“注入故障后 100 条零丢失”“切换 < 10 秒”。

一条消息从进入到投出去

【图片占位】

建议贴：When 全景架构图（接入/路由 → 时间轮 Master/Slave → Redis 落库 → 投递；ETCD 协调）

讲解重点：先落库再入内存——这是“不丢”的第一道保证，也是写进输入文档的硬约束。

案例

怎么拆：一块一份输入文档

接入层 + 接口契约

路由（一致性 hash）

多层时间轮引擎

存储层（Redis）

集群协调（ETCD）

副本与故障切换

投递与重试

拆的标准：一块能独立验收、20 分钟能写完输入文档。写不完，说明拆得太大。

案例

跑起来是什么样

交出文档

一次性把这块的输入交给循环

它自己转

写测试 → 实现 → 跑 → 修，反复

停下交接

全绿，或者卡住/有歧义时报告

我来判断

看思路、看边界、决定合并或改文档再跑

关键：不满意就改输入文档、从干净状态重跑，比在对话里跟它掰扯十回合有效得多。

同一个项目里，AI 的表现天差地别

- 这不是“AI 行不行”的问题，是“哪一类活它行”的问题。
- 同一个组件，有些模块它写得比我快也比我全；有些模块交给它就是埋雷。
- 分类标准只有一条：能不能廉价、确定地验证。

下面两页：具体点名——哪些它干得漂亮，哪些必须我自己定。

AI 干得漂亮的部分

多层时间轮算法

槽位计算、跨层降级、 $O(1)$ 增删——逻辑清晰，单测好写

存储 key 设计与读写

按 ID 拆小 key、批量管道、TTL 兜底

接口与序列化

契约定义、参数校验、错误码映射

投递插件与重试

HTTP / Kafka 实现 + 指数退避

共同点：输入输出明确、有确定答案、能被测试判定。这类活它又快又稳。

AI 容易翻车的部分

副本一致性

主写完、备没跟上，此时挂了怎么办

选主与脑裂

网络分区时，会不会同时出现两个主

故障切换时序

切换顺序、恢复顺序、窗口期的写入怎么处理

语义取舍

至少一次还是精确一次——这是产品决策，不是代码问题

共同点：只在“恰好那一刻”出错，普通测试全绿；代价高、事后难查。这类必须人定方案。

一条线：可验证性

交给循环

- 能廉价、确定地自动验证
- 错了立刻能发现、成本低
- 你的角色：定规格 + 审边界

你自己主导

- 难自动验证 + 错误代价高
- 涉及一致性、资金、安全
- AI 的角色：辅助分析与实现

怎么用这条线：拆任务时就把每块标上“谁主导”。别等写崩了才发现这块本来不该交出去。

经验：标错一块的代价，通常比自己写还大。

这套东西对一个人做项目意味着什么

- OPC（一人公司 / 超级个体）最缺两样东西：时间，和确定性。
- Loop Engineering 正好对上：把“执行”外包给循环，把时间还给判断。
- 但它放大的是你原本的判断质量——所以先说清楚，哪些东西它永远替不了。

顺序很重要：先讲提效在哪（真实的），再讲边界在哪（诚实的）。两个都说清，你才知道怎么用它。

放大明显的四件事

把清晰方案落成实现

方案想清楚了，实现这一段几乎是自动的

补测试与文档

过去最容易拖欠的，现在顺手就做了

读陌生代码 / 查资料

进入一个新领域的时间成本大幅下降

重构与样板

改结构、迁移、脚本这类活，成本接近零

体感：一个人确实能扛过去一个小团队的活——但前提是你把问题定义得足够清楚。

替不了的三样：判断力、思考、产品力

判断力

这个方案对不对、这个取舍值不值、
什么时候该停

思考

问题到底是什么、为什么是这个问题、
还有没有别的路

产品力

做给谁、他为什么要用、什么该砍、
什么绝不能砍

一个真实体会：做一个产品，最难的从来不是代码——是想清楚给谁做、为什么值得做、哪些功能必须砍掉。这部分没有任何工具能替你。

所以：AI 越强，你的判断力越值钱，因为它被放大了。

精力往哪放

70% 放在定义问题和验收

这两样定了，实现基本是自动的

按可验证性排任务顺序

先做能自动验证的，攒确定性

把难验证的部分做小

缩到一两块，集中精力攻

别追工具，追方法

工具半年一换，方法能用很久

一句话：一个人的瓶颈从来不是打字速度，是判断力和验证能力。

迁移

换成你的项目，这套怎么用

写业务系统

验收 = 接口契约 + 业务规则测试 + 端到端流程

做工具 / SaaS

验收 = 关键路径的自动化测试 + 数据正确性校验

做平台 / 产品

先想清楚给谁、砍什么（人），再拆成可验证的块（循环）

不变的部分：定义清楚 → 写可判定验收 → 让它跑 → 你守高风险决策。项目类型换了，方法不换。

三个常见误区

踩过才知道贵

“先跑起来再补测试”

循环没判官，越跑越偏，最后整块重来

“提示词写好一切都好”

提示词只影响单次输出，环境才决定长期产能

“有了 AI 就不用想清楚了”

恰恰相反：想不清楚，它会把模糊放大得更快

共同点：都想省掉“想清楚 + 设计验证”这一步，而这一步恰恰省不掉。

这周就能试的三件事

不用改工具链

挑一块小任务

选一个你手上能独立验收的小活

写一份输入文档

功能、接口、验收、边界，写到能判定

一次交出去，别插手

让它跑完再看，然后判断合并还是改文档重跑

跑完你会有个直观体感：卡住的地方，几乎都是文档没写清的地方。这就是下一次要改的东西。

五句话

- Loop Engineering = 设计一个有判官的循环，让 AI 自主干活；执行交给循环，判断留给人。
- 不用框架也能跑：SDD 思想定义清楚问题，Harness 思想搭好判官，极少交互交出去。
- 输入文档里最值钱的是“可判定的验收”——写不出来，说明还没想清楚。
- 边界只有一条线：能廉价确定验证的交循环，难验证又错不起的自己主导。
- 对 OPC 来说这是利器，但它放大的是你的判断力、思考和产品力——这三样只能是你自己的。

明天 (8/6)

用 AI 写复杂 infra 组件

- 案例：RobustMQ —— 用 Rust 做的消息队列，架构选型与技术取舍。
- 用 AI 写复杂 infra 组件的完整思路：怎么拆、怎么定契约、怎么验证正确性。
- 传统 infra 与 AI infra 的区别：新机会具体在哪一层。
- 开源社区正在发生什么，以及 Rust 的声音为什么越来越大。
- 最后回到个人：怎么把 infra 能力变成长期竞争力。

同一时间：明晚 8 点，第三场。

谢谢

AI + Loop Engineering 能极大提升产出，但判断力是你自己的

答疑时间 · 群内继续交流

【占位：二维码 / 课程与报名信息】